COMP2040: Operating Systems
Lab 02: Processes, Inter-Process Communication and Threads

Daniela Méndez Ramírez
College of Engineering and Computer Science
VinUniversity, Hanoi, Vietnam
Student ID: VU2614027

***Abstract*—This laboratory report explores process management, inter-process communication (IPC), and multithreading in POSIX operating systems using the C language. Five core concepts are evaluated: process creation and address space isolation with `fork()`, binary replacement via `execl()`, stream-based communication using unidirectional pipes, shared memory allocation with `mmap()`, and concurrent matrix multiplication using POSIX threads (`pthread`). The compilation commands, actual terminal execution outputs, and analytical answers are documented for each experimental task.**

***Index Terms*—Operating Systems, Processes, fork, exec, IPC, Pipes, Shared Memory, POSIX Threads, Concurrency.**

I. Introduction

Modern operating systems provide abstractions to run programs concurrently and protect resources. This lab turns Week 3 theoretical concepts into observable program behaviors.

The lab is divided into five practical tasks:

1) Creating a child process with `fork()` and verifying private virtual memory.
2) Replacing the child process image with `execl()` and monitoring its exit status.
3) Passing a string message from child to parent across a unidirectional pipe.
4) Comparing private stack variables against shared memory allocated with `mmap()`.
5) Dividing a 2×2 matrix multiplication between two concurrent threads using `pthread`.

In accordance with the lab submission guidelines, each task includes the compilation commands, textual terminal output, and concise answers to the required questions. Complete source code files are not embedded in the report.

All source code and related files are available in the following public repository:

https://github.com/Danym2407/Operative-Systems—Lab

II. Task 1: Create and Collect a Child Process

In this task, the system call `fork()` is used to create a child process. The child updates a local variable and prints its identifiers, while the parent waits for the child and verifies that its own copy remains unchanged.

A. Commands and Observed Output

The program `task1_fork.c` was compiled with strict flags and executed in the terminal:

```
$gcc -std=c11 -Wall -Wextra task1_fork.c -o
    task1_fork$ ./task1_fork
child: PID=54322 PPID=54321 value=99
parent: PID=54321 value=10
```

B. Analysis and Questions

- **(a) Compilation command and output lines:** The program was compiled with `gcc -std=c11 -Wall -Wextra task1_fork.c -o task1_fork`. The execution produced two distinct lines showing the child process outputting `value=99` followed by the parent process reporting `value=10`.
- **(b) Return values of `fork()`:** In the parent process, `fork()` returns the positive PID of the newly created child process. Inside the child process, `fork()` returns `0`. If process creation fails, it returns `-1` and sets the global variable `errno`.
- **(c) Memory isolation between processes:** Modifying `value` in the child did not alter the parent's value because `fork()` creates a separate, duplicated virtual address space for the child process. Both processes have their own private copy of the variable stored at different physical memory locations under copy-on-write semantics.
- **(d) Purpose of `waitpid()`:** The `waitpid()` call blocks the parent process so it does not print its output or exit before the child finishes. Furthermore, it harvests the child's termination state, preventing the terminated child from remaining in the system process table as a zombie process.

III. Task 2: Replace the Child's Program

This task demonstrates how a child process can execute a completely separate binary (`task2_worker`) using `execl()`, while the parent monitors its termination status.

A. Commands and Observed Output

Both programs were compiled in the same directory, followed by the execution of `task2_fork_exec`:

```
$ gcc -std=c11 -Wall -Wextra task2_worker.c -o
    task2_worker
$gcc -std=c11 -Wall -Wextra task2_fork_exec.c -o
    task2_fork_exec$ ./task2_fork_exec
before exec: PID=71240
worker: PID=71240 word=COMP2040
parent: child exited with status 0
```

B. Analysis and Questions

- **(a) Compilation commands and program output:** Both programs were compiled without warnings using the C11 standard. The terminal output confirmed that the child printed its PID before replacement, the worker executed using that same PID, and the parent reported exit status `0`.
- **(b) Process creation during `execl()`:** The `execl()` function did not create another process. The PID printed immediately before `execl()` (`PID=71240`) is identical to the PID printed inside `task2_worker`. The call simply replaced the process memory space while preserving its existing process ID.
- **(c) Execution of `_exit()` after `execl()`:** When `execl()` succeeds, the existing code and address space are overwritten by the new program, so the remaining lines are never reached. The child process only reaches the subsequent `_exit()` if `execl()` fails and returns `-1`.
- **(d) Difference between `fork()` and `exec()`:** The `fork()` system call duplicates an existing process and assigns a brand-new PID to the child. In contrast, `exec()` replaces the code, data, heap, and stack of the current process with a new binary, executing it under the exact same PID.

IV. Task 3: Send a Message Through a Pipe

This task implements one-way inter-process communication using a kernel pipe, sending a text string from the child writer to the parent reader.

A. Commands and Observed Output

The source file `task3_pipe.c` was compiled and executed:

```
$gcc -std=c11 -Wall -Wextra task3_pipe.c -o
    task3_pipe$ ./task3_pipe
Parent received: Hello from the child
```

B. Analysis and Questions

- **(a) Compilation command and received message:** The code compiled cleanly using `gcc -std=c11 -Wall -Wextra task3_pipe.c -o task3_pipe`. The parent successfully received and printed the exact message: `Parent received: Hello from the child`.
- **(b) Pipe descriptors and closure policy:** The child writes to `fd[1]` and must close its unused read descriptor `fd[0]`. The parent reads from `fd[0]` and must close its unused write descriptor `fd[1]`. Closing unused ends ensures proper end-of-file (EOF) detection and avoids hanging read calls.
- **(c) Reading child data despite private memory:** Unlike the private variable in Task 1, a pipe does not reside in the user-space virtual memory of either process. A pipe is a circular buffer allocated and managed inside the operating system kernel. Both processes access this shared kernel buffer through their inherited file descriptors.
- **(d) Pipe directionality and two-way communication:** The pipe created by `pipe()` is strictly one-way (unidirectional). To establish two-way communication, the application must create two separate pipes: one dedicated to child-to-parent data transmission, and a second dedicated to parent-to-child transmission.

V. Task 4: Compare Private and Shared Memory

This task compares process-isolated stack memory against explicit inter-process shared memory allocated through the `mmap()` system call.

A. Commands and Observed Output

The program `task4_shared_memory.c` was compiled and run:

```
$gcc -std=c11 -Wall -Wextra task4_shared_memory.c -
    o task4_shared_memory$ ./task4_shared_memory
parent: private=10 shared=99
```

B. Analysis and Questions

- **(a) Compilation command and output:** The binary was compiled using `gcc -std=c11 -Wall -Wextra task4_shared_memory.c -o task4_shared_memory`. The parent output printed `private=10 shared=99`.
- **(b) Reason only the shared value changed in the parent:** The variable `private_value` lives on the process stack and was duplicated into a private address space during `fork()`. In contrast, `shared_value` points to a physical memory region mapped with `MAP_SHARED`, meaning the page tables of both processes map to the same physical RAM frame.
- **(c) Shared memory vs. pipe IPC:** Pipes require system calls (`read()` and `write()`) where the kernel copies data between user space and kernel space buffers. Shared memory maps RAM directly into both processes' virtual spaces, allowing memory reads and writes with zero copying and without kernel intervention on each operation.
- **(d) Simultaneous write hazards:** If both processes wrote to the shared integer simultaneously without synchronization, a race condition would occur. Without mutual exclusion primitives such as mutexes or semaphores, concurrent unsynchronized writes can cause lost updates or corrupted data.

VI. Task 5: Compute a Matrix with Two Threads

This task uses POSIX threads (`pthread`) to parallelize a 2×2 matrix multiplication:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$

Each thread computes exactly one row of the output matrix.

A. Commands and Observed Output

The code was compiled linking the pthread library, producing the expected dot-product rows:

```
$gcc -std=c11 -Wall -Wextra -pthread task5_threads.
    c -o task5_threads$ ./task5_threads
19 22
43 50
```

B. Analysis and Questions

- **(a) Compilation command and resulting matrix:** Compiled with `gcc -std=c11 -Wall -Wextra -pthread task5_threads.c -o task5_threads`. The resulting output matrix matched the expected mathematical values: row 0 produced `19 22` and row 1 produced `43 50`.
- **(b) Shared data and thread work assignment:** The threads share the global address space of the process, including read-only matrices `a` and `b`, and the shared array `result`. Thread 0 is assigned row 0 and computes elements $(0, 0)$ and $(0, 1)$, while thread 1 is assigned row 1 and computes elements $(1, 0)$ and $(1, 1)$.
- **(c) Necessity of `pthread_join()`:** The main thread must call `pthread_join()` to block until both worker threads finish their computations. If the main thread proceeded without joining, it would print the matrix before the workers wrote their values, outputting uninitialized or incomplete data.
- **(d) Benefit and cost of processes versus threads:** One benefit of using processes is fault isolation; if a child process crashes, it does not terminate the rest of the application. One major cost is higher overhead in memory consumption and context switching, along with the requirement to set up explicit IPC mechanisms like `mmap()` to share the output array.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.

[2] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco, CA: No Starch Press, 2010.

[3] Linux man-pages project, "fork(2) – create a child process," Linux manual page.

[4] Linux man-pages project, "exec(3) – execute a file," Linux manual page.

[5] Linux man-pages project, "pipe(2) – create pipe," Linux manual page.

[6] Linux man-pages project, "mmap(2) – map files or devices into memory," Linux manual page.

[7] Linux man-pages project, "pthread_create(3) – create a new thread," Linux manual page.